

Подготовка к экзамену по C++ (ИТМО)

Тема 7: STL. Ассоциативные контейнеры (Деревья и Хэши)

Hardcore Theory Edition

17 июня 2026 г.

1 Введение: Что такое ассоциативные контейнеры?

В `vector` мы обращаемся к элементам по их абсолютному номеру (индексу). В ассоциативных контейнерах индексов нет. Мы обращаемся к данным по **Ключу**. Здесь правят бал два семейства: **Упорядоченные** (Деревья) и **Неупорядоченные** (Хэш-таблицы).

2 Семейство Деревьев: `std::map` и `std::set`

2.1 Под капотом: Красно-черное дерево (Red-Black Tree)

`std::map` не имеет ничего общего с массивами. Внутри памяти он представляет собой самобалансирующееся бинарное дерево поиска.

- **Аллокация:** Каждый элемент (узел дерева) выделяется в куче отдельным вызовом `new`. Узел хранит указатели на `left`, `right`, `parent`, свой цвет (красный/черный) и саму пару `<const Key, Value>`. Это означает **огромный расход памяти** и плохую работу с кэшем процессора.
- **Неизменяемость ключей:** Ключ всегда константный (`const Key`). Если бы вы могли изменить ключ напрямую, дерево бы сломалось (потеряло сортировку). Чтобы изменить ключ, нужно удалить старый узел и вставить новый.
- **Скорость:** Поиск, вставка и удаление всегда занимают $O(\log N)$.

Суть на пальцах (Жизненная аналогия)

`std::map` — это картотека в регистратуре поликлиники. Все карточки строго отсортированы по алфавиту (по фамилиям). Когда вы ищете "Яковлева" вы не перебираете всех подряд. Вы открываете ящик посередине, видите букву "М" понимаете, что "Я" дальше, открываете середину правой половины и так далее. Это логарифмический поиск.

2.2 Требования к Ключу

Чтобы дерево могло сортировать элементы, оно должно уметь их сравнивать. По умолчанию используется `operator<`. Если вы хотите использовать свою структуру как ключ, вы **обязаны** перегрузить оператор `<`.

```

1 struct Point { int x, y; };
2
3 // Без этого оператора std::map<Point, std::string> не скомпилируется!
4 bool operator<(const Point& a, const Point& b) {
5     if (a.x != b.x) return a.x < b.x;
6     return a.y < b.y;
7 }

```

3 Семейство Хэшей: `std::unordered_map`

Появилось только в C++11. Это классическая хэш-таблица. Элементы внутри лежат **хаотично**, никакой сортировки нет.

3.1 Под капотом: Массив корзин (Buckets) и Хэш-функция

Как работает `unordered_map` при вставке элемента `["Alice": 25]`:

1. Строка "Alice" передается в хэш-функцию (`std::hash`). На выходе получается гигантское псевдослучайное число (например, 15498416515).
2. Это число берется по модулю размера внутреннего массива (количества корзин). Например, $15498416515 \% 10 = 5$.
3. Значение кладется в массив под индексом 5.

Скорость: В идеальном мире это занимает $O(1)$. Мгновенный доступ!

3.2 Проблема Коллизий и Инвалидация итераторов

Что если для строки "Bob" хэш тоже укажет на индекс 5? Это **Коллизия**. В STL коллизии разрешаются методом цепочек (Separate Chaining). В ячейке номер 5 лежит не сам элемент, а указатель на односвязный список (`std::forward_list`). Туда просто добавится новый узел.

Рехэширование (Rehashing): Когда элементов становится больше, чем корзин (Load Factor > 1.0), списки становятся слишком длинными, и скорость падает до $O(N)$. Тогда таблица выделяет новый, БОЛЬШОЙ массив, и **пересчитывает хэши для ВСЕХ элементов**. *Важно на экзамен:* Рехэширование инвалидирует ВСЕ итераторы!

Суть на пальцах (Жизненная аналогия)

`unordered_map` — это огромный склад почты с корзинами по почтовым индексам (хэшам). Работник просто смотрит на индекс письма и не глядя кидает его в нужную корзину за секунду ($O(1)$). Но если все письма вдруг пришли на один индекс, одна корзина переполнится, и работнику придется рыться в ней полчаса (Коллизия, $O(N)$).

3.3 Требования к Ключу

Для `unordered_map` оператор `<` не нужен! Деревя ведь нет. Вместо этого компилятору нужны две вещи:

1. `operator==` (чтобы понимать, нашли ли мы точный элемент внутри корзины при коллизии).

2. Специализация структуры `std::hash<ТвойТип>` (чтобы компилятор знал, как превратить твой класс в число).

4 Ловушка Джокера: оператор []

Любимый вопрос преподавателей. В `std::map` и `unordered_map` оператор квадратных скобок `map[key]` работает очень коварно.

Если ключа нет в словаре, оператор [] АВТОМАТИЧЕСКИ СОЗДАСТ ЕГО, вызвав конструктор по умолчанию для Value!

```
1 std::map<std::string, int> m;
2 m["Alice"] = 10;
3
4 // Мы просто хотели проверить, есть ли Bob...
5 if (m["Bob"] == 0) {
6     std::cout << "Bob is not here.\n";
7 }
8
9 std::cout << m.size();
10 // Выведет 2 !!! Мы случайно создали Bob'а с нулевым балансом.
11
12 // ПРАВИЛЬНЫЙ СПОСОБ ПРОВЕРКИ:
13 if (m.find("Bob") != m.end()) { /* ... */ }
```